

Smart Construction Connect

Infrastructure, storage architecture and containerisation handbook — everything needed to build images and deploy this application on Kubernetes / EKS.

Repository: <https://github.com/MuhammadAzan169/Smart-Construction-Connect> · **Branch:** main · **Revision:** 2 · **Date:** 5 September 2026

Prepared for: DevOps containerisation / EKS practice deployment. Every answer below was verified against the source code and by running both services locally; file and line references are given throughout.

BLOCKER — FIX BEFORE DEPLOYING

The Supabase project referenced by the current configuration no longer exists. Both `DATABASE_URL` and `SUPABASE_URL` point at project ref `rvezbjvbwrnsnopmaftna`, whose hostname does not resolve in DNS and whose pooler returns `FATAL: (ENOTFOUND) tenant/user not found`. A new Supabase project must be created and those variables updated, otherwise the application has no durable storage. See section 4.

CHANGED SINCE REVISION 1

The repository directories are now lowercase (`backend/`, `frontend/`). Container definitions now **exist in the repository** — `backend/Dockerfile` and `frontend/Dockerfile` — so section 6 documents the real files instead of reference ones. The backend now refuses to start in production when the database is unreachable, rather than silently falling back to local files.

Contents

1. What the application is
2. Current production deployment
3. Storage architecture — the four directories answered (A–E)
4. What this means for EKS: EBS vs S3 vs Supabase vs ephemeral
5. Runtime characteristics that affect Kubernetes
6. The container images in this repository
7. Environment variables reference
8. Networking, routing and ingress
9. Health checks, probes and resources
10. Known gotchas

1. What the application is

A construction marketplace and AI assistant platform. Clients, construction companies, material suppliers and administrators each get a role-scoped dashboard. Features: JWT authentication, company/supplier discovery and verification, project requests, real-time chat, document upload with text extraction, an English/Urdu bilingual UI, and a retrieval-augmented AI assistant.

It is a **two-part application in one repository**:

Part	Technology	Description
frontend/	React 18, TypeScript, Vite 7, Tailwind CSS, shadcn/ui. Node 22 to build.	Static single-page application. <code>vite build</code> emits a folder of static files — no Node runtime in production, only a web server.
backend/	Python 3.11.9, FastAPI, Uvicorn	Stateless HTTP API under <code>/api/...</code> , plus one WebSocket endpoint and one Server-Sent Events endpoint. Single ASGI process.

There is no separate worker, queue, cron job or cache service. The full runtime is one Python process, one static file server, and the managed services below. A root-level `app.py` exists purely as a local convenience runner (it installs dependencies and starts both dev servers); it is **not** a deployment entry point.

2. Current production deployment

Component	Platform	Details
Frontend SPA	Vercel	Root directory <code>frontend/</code> , build <code>vite build</code> , output <code>dist/</code> . Live at <code>smart-construction-connect-frontend.vercel.app</code>
Backend API	Render (free web service)	Root directory <code>backend/</code> , start <code>uvicorn app:app --host 0.0.0.0 --port \$PORT</code> . Live at <code>smart-construction-connect-backend.onrender.com</code>
Database	Supabase Postgres	Transaction pooler (pgBouncer), port 6543. One table: <code>documents(key text pk, data jsonb, updated_at timestampz)</code>
File storage	Supabase Storage	One public bucket, default <code>media</code> , via its REST API with the service-role key.
Embeddings	Cohere	<code>api.cohere.ai/compatibility/v1</code> , model <code>embed-multilingual-light-v3.0</code> . Optional — falls back to BM25.
LLM chat	OpenRouter	Free model chain with automatic key and model rotation on rate limits.

All six are reachable over plain HTTPS, so **the same architecture works unchanged inside Kubernetes** — pods need outbound internet (NAT gateway) and the secrets injected as environment variables.

3. Storage architecture — the four directories answered

The backend behaves differently depending on which environment variables are set. This is the single most important thing to understand before deploying it:

Configuration	JSON documents go to	Binary files go to
DATABASE_URL and SUPABASE_* empty (local development)	Local files under backend/Database/	Local disk under backend/company_data/ and backend/uploads/
All three set (production — deploy this way)	Supabase Postgres, table documents	Supabase Storage bucket

The switch is automatic and checked on every read and write — there is no feature flag to enable.

A. Is Database/*.json still the actual production database?

No — not in production. It is the local-development store and the migration source. In production every read and write goes to Supabase Postgres.

All JSON access funnels through two functions, which check Postgres before touching disk:

```
def read_json(filepath: Path) -> Any:
    from backend.utils import db
    if db.is_enabled():
        # DATABASE_URL set and connected
        val = db.kv_get(_doc_key(filepath)) # SELECT data FROM documents WHERE key = %s
        return val if val is not None else []
    ... # otherwise read the local file

def write_json(filepath: Path, data: Any):
    from backend.utils import db
    if db.is_enabled():
        db.kv_set(_doc_key(filepath), data) # INSERT ... ON CONFLICT DO UPDATE
        return
    ... # otherwise atomic temp-file + os.replace
```

Source: backend/backend/utils/data_handler.py lines 89-132; backend/backend/utils/db.py.

The Postgres key is the document's old file path — Database/clients/users.json becomes one row whose data column holds the whole JSON array.

Consequence: the Database/ directory in the image holds seeded demo data. In production it is never written to. **No persistent volume required.**

B. Does company_data/ still serve files directly from the container?

Partly — and this is the one real subtlety.

- **Demo assets committed to the repository** (logos, galleries, sample documents) are still served by the container as static files:

```
COMPANY_DATA_DIR = ROOT / "company_data"
app.mount("/company_data", StaticFiles(directory=str(COMPANY_DATA_DIR)), name="static-company-data")

UPLOADS_DIR = ROOT / "uploads"
app.mount("/uploads", StaticFiles(directory=str(UPLOADS_DIR)), name="static-uploads")
```

Source: backend/app.py lines 193-202. They are read-only, come from the git checkout, and are identical in every replica — **image content, not state.**

- **New uploads** do not land there in production. They go to Supabase Storage, and the API returns an absolute CDN URL the browser fetches directly, bypassing the backend.

```
def save_bytes(rel_path: str, data: bytes, content_type: str = "...") -> str:
    if is_remote():
        # SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY both set
        endpoint = f"{url}/storage/v1/object/{bucket}/{rel_path}"
        resp = httpx.post(endpoint, headers={"Authorization": f"Bearer {key}", ...}, content=data)
        return f"{url}/storage/v1/object/public/{bucket}/{rel_path}" # absolute CDN URL
    dest = _REPO_ROOT / rel_path # local disk fallback (development only)
    dest.write_bytes(data)
    return f"/{rel_path}"
```

Source: `backend/backend/utils/storage.py` lines 41-78.

Consequence: keep the `/company_data` route working (old records still hold relative URLs), but it needs **no shared or persistent volume**.

C. Does `uploads/` receive user uploads during runtime?

Only in local development. In production, attachments and voice notes go to Supabase Storage through the same `save_bytes()`.

The directory is created at startup and mounted so development works and legacy relative URLs still resolve. `uploads/*` is git-ignored, so it ships empty apart from a marker file. An index of uploads is kept as a JSON document (`Database/uploads_index.json`) so the admin file monitor works when binaries live in Supabase — meaning it too lives in Postgres in production.

Consequence: ephemeral filesystem is correct. No volume needed.

D. Is `index_data/` just a rebuildable cache?

Yes. Entirely rebuildable derived data — safe to lose at any time.

It holds two artefacts, `faiss.index` and `meta.pkl` (BM25 state, entity records, content hashes), both derived from company and supplier data. Git-ignored, and built during startup:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    count = initialize_embeddings() # builds the hybrid semantic index
    response_cache.invalidate()
    stop_watcher = start_background_watcher() # polls source data every 10s
    yield
    stop_watcher()
```

Source: `backend/app.py`; `backend/backend/utils/semantic_embeddings.py` lines 47-49; `backend/backend/utils/index_sync.py`.

Measured locally: a full build over 99 companies and 24 suppliers (123 vectors) took **19.5 seconds**.

Consequence: ephemeral filesystem, and never share it between pods. Each replica builds its own copy. Sharing a FAISS index and a pickle over a network volume invites corruption for no benefit.

E. Which of these have already moved to Supabase?

Directory	Production destination	Written at runtime?	Volume required
Database/	Supabase Postgres	Yes — to Postgres, not disk	None
company_data/	New uploads to Supabase Storage ; demo assets from the image	No (read-only in production)	None
uploads/	Supabase Storage	No	None
index_data/	Local, rebuilt per pod at startup	Yes, disposable cache only	None (ephemeral, per-pod)

4. What this means for EKS: EBS vs S3 vs Supabase vs ephemeral

SHORT ANSWER

Ephemeral filesystem. No EBS, no EFS, no S3, no PersistentVolumeClaim. Provided the three Supabase variables are set, all durable state lives in Supabase and the backend pod is stateless. Use a plain `Deployment`; a `StatefulSet` would buy nothing.

Option	Needed?	Reasoning
EBS	No	Nothing durable is written to local disk. EBS is also single-attach (<code>ReadWriteOnce</code>), capping the deployment at one replica per volume for no gain.
EFS	No	Only relevant if pods had to share writable state. Postgres and Supabase Storage already do that.
S3	No, but a good exercise	Supabase Storage already handles binaries. The swap is well isolated: <code>storage.py</code> is ~114 lines with two functions, <code>save_bytes()</code> and <code>delete_by_url()</code> . Reimplement those with <code>boto3</code> and nothing else changes. Prefer IRSA over static keys.
Supabase	Yes — keep it	The durable tier, reachable over HTTPS from anywhere. Pods need outbound internet only.
Ephemeral	Yes — this is the answer	The container filesystem holds only code, demo assets, and the rebuildable index.

Creating the Supabase project (currently required)

The configured project no longer exists, so this must be done before any deployment expecting persistence:

1. Create a project at supabase.com.
2. **Database** → **Connect** → copy the **Transaction pooler** URI (host `...pooler.supabase.com`, port `6543`) and put the DB password in it → `DATABASE_URL`.
3. **Settings** → **API** → copy the Project URL (`SUPABASE_URL`) and, from the "Legacy anon, service_role API keys" tab, the `service_role` key (a JWT starting `eyJ...`). The newer opaque `sb_secret_...` format is rejected by

Supabase Storage with "Invalid Compact JWS".

4. **Storage** → create a **public** bucket named `media`.

5. Run `python migrate_to_supabase.py --dry-run`, then without the flag.

Running with no external dependencies at all

Leaving `DATABASE_URL` and `SUPABASE_*` **empty** makes the app use local JSON files and local disk — a legitimate way to demo it in-cluster without a Supabase account. It then becomes stateful: you would need a ReadWriteOnce PVC and exactly one replica. Alternatively run Postgres in-cluster; nothing in the data layer is Supabase-specific (section 7).

IMPORTANT DISTINCTION

An **empty** `DATABASE_URL` is fine and starts normally. A `DATABASE_URL` that is *set but unreachable* **aborts startup** when `ENV=production` — the container exits and Render reports "Exited with status 3". Do not point it at a dead project expecting a silent fallback. To deploy anyway on throwaway storage, set `ALLOW_EPHEMERAL_STORAGE=1`, which downgrades the refusal to a loud warning (gotcha 1).

5. Runtime characteristics that affect Kubernetes

The backend holds in-memory state that is not shared between replicas

The backend is stateless with respect to *durable data*, but three pieces of live state exist only inside a single process:

In-memory state	Where	Effect with multiple replicas
WebSocket connection registry — <code>dict[email, list[WebSocket]]</code>	<code>routers/messages.py</code> line 39	A message is pushed in real time only to sockets connected to <i>the same pod</i> . With two replicas, two users on different pods will not see each other's messages arrive live — they are still stored correctly and appear on refresh.
SSE event bus — <code>list[asyncio.Queue]</code> of subscribers	<code>utils/events.py</code> line 31	Admin live-monitoring events reach only SSE clients attached to the pod that produced the event.
Semantic index, response cache, conversation memory	<code>utils/semantic_embeddings.py</code> , <code>response_cache.py</code> , <code>conversation_memory.py</code>	Harmless duplication — each pod builds its own. Costs memory and startup time, nothing more.

RECOMMENDATION

Start with **one backend replica**. It is the honest configuration for this codebase and everything works. To scale out, in increasing order of effort: (1) accept degraded cross-pod real-time chat; (2) enable session affinity on the ingress so one user's HTTP and WebSocket land on one pod — fixes the common case but not delivery between pods; (3) replace the in-process fan-out in `messages.py` and `events.py` with Redis pub/sub, which touches only those two files and is the genuinely instructive version.

Other runtime facts

- **Port:** the app reads `$PORT` and falls back to `8000`.

- **Single process:** the start command runs one Uvicorn worker. Do not add `--workers N` without reading the note above — each worker is a separate process with its own connection registry and index, so it has the same split-brain effect as extra replicas, at $N \times$ the memory.
- **Startup work:** the semantic index is built during lifespan startup, before traffic is served. With an embeddings API configured this means one outbound call per company/supplier at boot (19.5s measured locally).
- **Background thread:** a daemon thread polls source data every 10 seconds and incrementally refreshes the index. It stops cleanly on shutdown, so `SIGTERM` handling is fine; the default 30s `terminationGracePeriodSeconds` is plenty.
- **No local ML in the deployed image:** PyTorch, sentence-transformers, FAISS and EasyOCR are kept out of `requirements.txt` so the service fits in 512 MB. They live in `requirements-local.txt` for offline development only.

Measured latencies (local, warm backend)

Operation	Time	Note
Semantic index build at startup	19.5 s	Before the app accepts traffic — size your startup probe for it
AI chat, streaming (<code>/api/ai/chat/stream</code>)	0.4 s to first token, 13.6 s total	What the UI actually uses
AI chat, non-streaming (<code>/api/ai/chat</code>)	42.9 s	Only used for the canned greeting, which returns instantly
Chat with file upload (<code>/api/ai/chat-with-file</code>)	27.2 s	Document extraction plus LLM; proxy timeouts must exceed this

These numbers are from a developer machine. Render's free CPU is slower, and a cold start adds 30-60 s on top. Every proxy, ingress and client timeout must be sized against the slow case, not this table.

6. The container images in this repository

Both parts now ship a Dockerfile. Neither has been built in this environment (no Docker daemon was available), so treat the first build as the verification step.

Backend — `backend/Dockerfile`

Python 3.11.9 slim base, dependency layer cached ahead of the code copy, build tools purged in the same layer, runs as a non-root user (UID 10001), and honours `$PORT`:

```
docker build -t scc-api ./backend
docker run --rm -p 8000:8000 --env-file backend/.env scc-api
curl http://localhost:8000/api/health # {"status":"ok","version":"1.0.0"}
```

Two things to know about this image:

- It sets `ENV=production` in the image. Combined with the startup guard, a container given a *broken* `DATABASE_URL` will refuse to boot. For a dependency-free trial run, either leave `DATABASE_URL` empty (cleanest) or set `ALLOW_EPHEMERAL_STORAGE=1` to override the guard.

- It installs only `requirements.txt`. `requirements-local.txt` (PyTorch, FAISS, EasyOCR) is excluded by `.dockerignore` so it cannot accidentally bloat the image.

Frontend — `frontend/Dockerfile`

Two stages: Node 22 builds the bundle, then unprivileged nginx serves it. There is no Node runtime in the final image.

```
docker build -t scc-web --build-arg VITE_API_URL=https://api.example.com ./frontend
docker run --rm -p 8080:8080 -e BACKEND_ORIGIN=http://host.docker.internal:8000 scc-web
```

Setting	When it applies	Notes
<code>VITE_API_URL</code>	Build time	Vite inlines <code>VITE_*</code> into the bundle. Changing it requires a rebuild. Only needed when the browser reaches the backend on a different origin than the page — it is what the chat WebSocket dials. Leave empty when serving the API through this container's own proxy.
<code>BACKEND_ORIGIN</code>	Run time	Origin that <code>/api/</code> , <code>/uploads</code> and <code>/company_data</code> proxy to, substituted into the nginx config at container start. No trailing slash — nginx would treat it as a URI part and strip the path prefix.

The nginx config (`frontend/nginx.conf.template`) reproduces what `vercel.json` does in the Vercel deployment, plus the pieces a proxy needs that Vercel handles implicitly:

- **SPA fallback** — unknown paths return `index.html` so client-side routing works.
- **WebSocket upgrade** on `/api/`, with a `map` for `$connection_upgrade` and a 3600 s read timeout, so idle chat sockets are not cut.
- **proxy_buffering off** — required for both the token-by-token AI stream and the SSE admin feed. With buffering on, replies arrive in bursts or not at all.
- **client_max_body_size 12m** — above the 10 MB server-side document limit, so uploads are not rejected with a 413 at the proxy.
- **Cache headers** — hashed assets under `/assets/` are immutable for a year; `index.html` is `no-store`, or clients keep loading the previous build's asset URLs after a deploy.

It listens on **8080** as UID 101, so the pod needs no root and no `NET_BIND_SERVICE` capability.

Two routing models — pick one

- **Same-origin (recommended, mirrors production).** One ingress host. Either let the frontend container proxy `/api` itself via `BACKEND_ORIGIN`, or route those paths at the ingress. No CORS configuration, and the WebSocket travels over the same host. Leave `VITE_API_URL` empty.
- **Split origins.** Frontend and backend on separate hosts. Then you *must* set `CORS_ORIGINS` on the backend to the frontend's exact origin, and build the frontend image with `VITE_API_URL` so the WebSocket can find the backend.

7. Environment variables reference

Everything is read from the environment; nothing is hardcoded. `.env` files are git-ignored and have never been committed, so the repository contains no credentials — you will need your own Supabase project and API keys,

or you can run in local-JSON mode with none of them.

Backend

Variable	Secret	Required	Notes
ENV	No	Yes	<code>production</code> enforces a strong <code>JWT_SECRET_KEY</code> , disables autoreload, and turns an unreachable database into a startup failure. Already baked into the backend image.
PORT	No	No	Defaults to 8000.
JWT_SECRET_KEY	Yes	Yes	<code>python -c "import secrets; print(secrets.token_hex(32))"</code> . Must be identical across replicas or tokens break on every pod hop.
CORS_ORIGINS	No	Conditional	Comma-separated exact origins. Only needed when the browser calls the API on a different origin than the page.
DATABASE_URL	Yes	For durability	Supabase transaction-pooler URI (<code>...pooler.supabase.com:6543</code>). Empty means local JSON. Set-but-broken aborts startup in production.
SUPABASE_URL	No	For durability	<code>https://<project-ref>.supabase.co</code>
SUPABASE_SERVICE_ROLE_KEY	Yes	For durability	The <i>legacy</i> <code>service_role</code> JWT (<code>eyJ...</code>). Backend only — never expose to a browser.
SUPABASE_BUCKET	No	No	Defaults to <code>media</code> . Must exist and be public.
OPENROUTER_API_KEY1...N	Yes	For AI chat	Every variable starting <code>OPENROUTER_API_KEY</code> is auto-discovered and rotated on rate limits.
EMBEDDINGS_API_URL / _KEY / _MODEL	Key: yes	No	Currently Cohere. Omit entirely and search degrades to BM25 keyword matching — the app still works and costs nothing.
ALLOW_EPHEMERAL_STORAGE	No	No	Set to <code>1</code> to start in production even when <code>DATABASE_URL</code> is unreachable, using local-file storage that is lost on every restart. Demos only. Unset (the default) means an unreachable database aborts startup.
DB_POOL_MAX	No	No	Pool ceiling, default 5. Total connections are replicas × this value.
LOG_LEVEL	No	No	<code>INFO</code> by default.

Put the four secrets in a Kubernetes `Secret` (or External Secrets / AWS Secrets Manager) and the rest in a `ConfigMap`. Documented defaults live in `backend/.env.example`.

Frontend

Variable	Secret	When	Notes
VITE_API_URL	No — public	Build time	The browser-reachable backend origin, used by the chat WebSocket. Required only for split-origin deployments. Leave unset for same-origin routing and for local development, where the Vite dev server proxies the socket.

IMPORTANT

Anything prefixed `VITE_` is compiled into the JavaScript bundle and is **publicly visible to anyone who opens the site**. Never put a secret in a `VITE_` variable.

A note on non-Supabase Postgres

Nothing in the data layer is Supabase-specific — it is plain Postgres with a single JSONB table the app creates itself on boot. Point `DATABASE_URL` at RDS or an in-cluster Postgres and it works. One detail: the connection sets `prepare_threshold = None` to disable server-side prepared statements for pgBouncer transaction-mode compatibility, which is harmless (very slightly slower) on a direct connection.

8. Networking, routing and ingress

Routes exposed by the backend

Path	Type	Purpose
/api/auth/*	HTTP	Signup, login, JWT issuance, profile, password change
/api/companies/*, /api/suppliers/*, /api/requests/*	HTTP	Marketplace and project requests
/api/admin/*	HTTP	Admin console (guarded by an admin-role dependency)
/api/analytics/*	HTTP	Admin analytics (also admin-only)
/api/ai/*	HTTP + SSE	AI assistant, including /api/ai/chat/stream . The slowest endpoints.
/api/upload/*	HTTP	File uploads — needs a body-size limit of at least 10 MB
/api/messages/*	HTTP	Conversations and message history
/api/messages/ws/{email}	WebSocket	Real-time chat, authenticated by a ?token= query parameter
/api/events/stream	SSE	Admin live monitoring — a long-lived streaming response
/api/health	HTTP	Health check — {"status":"ok","version":"1.0.0"}
/docs	HTTP	Swagger UI. Consider blocking at the ingress in a public deployment.
/company_data/*, /uploads/*	Static	Legacy and demo assets served from the image

Ingress settings that actually matter

- **WebSocket upgrade** must be allowed on /api/messages/ws/. AWS ALB supports this natively; with the nginx ingress controller it works by default, but raise proxy-read-timeout (e.g. 3600) or idle chat sockets get cut.
- **SSE and streaming must not be buffered.** Both /api/events/stream and /api/ai/chat/stream need proxy_buffering off; otherwise AI replies arrive in one lump at the end instead of streaming, which users read as a hang.
- **Upload body size:** defaults are 5 MB for images and 10 MB for documents. nginx's 1 MB default rejects uploads with a 413.
- **Timeouts:** a file-chat request measured 27.2 s locally and a cold backend adds 30-60 s. A 30 s proxy timeout will produce spurious 504s; 120 s is a safer floor.
- **Session affinity** is only relevant with more than one replica — see section 5.

Egress

The backend pod needs outbound HTTPS to Supabase (Postgres on 6543 and Storage on 443), openrouter.ai, and api.cohere.ai. In a private-subnet EKS setup that means a NAT gateway. If you apply NetworkPolicies, allow those destinations or the app will boot but every AI and persistence call will fail.

9. Health checks, probes and resources

Probe	Endpoint	Suggested settings
Liveness	GET /api/health	periodSeconds: 20, failureThreshold: 3
Readiness	GET /api/health	periodSeconds: 10, failureThreshold: 3
Startup	GET /api/health	failureThreshold: 30, periodSeconds: 5 — allows up to 150 s for the index build

WHY A STARTUP PROBE MATTERS HERE

The semantic index is built during FastAPI's lifespan startup, *before* the server accepts requests — 19.5 s locally, longer on a small node, and longer again when an embeddings API is called once per company and supplier. Without a startup probe, an aggressive liveness probe can kill the pod mid-build and produce a restart loop that never completes.

Resource sizing

Workload	Requests	Limits	Notes
Backend	200m CPU / 512Mi	1000m CPU / 1Gi	Runs today within Render's 512 MB free tier, so 1Gi is comfortable headroom. Go well above 2Gi only if you re-enable the local ML extras.
Frontend (nginx)	50m CPU / 64Mi	200m CPU / 128Mi	Static file serving plus a proxy — very light.

The frontend is genuinely stateless and horizontally scalable; put an HPA on it freely. Apply the section 5 caveats before scaling the backend.

10. Known gotchas

1. **A broken database URL used to fail silently.** If `DATABASE_URL` was set but unreachable, the app logged an error and carried on using local JSON files — which on an ephemeral pod looks like it is working while losing every write on restart. It now **refuses to start** when `ENV=production`: the process raises during FastAPI lifespan startup, uvicorn logs "Application startup failed. Exiting.", and Render reports `Exited with status 3` with no open ports. That is the guard working, not a separate bug — the fix is a reachable database. If you deliberately want throwaway storage (a demo with no Supabase project), set `ALLOW_EPHEMERAL_STORAGE=1` and it starts with a loud warning instead. In development it still falls back automatically. Either way, check the startup logs for "Database enabled - using Postgres document store" before believing persistence is active.
2. **Storage has the same shape of failure.** If the Supabase variables are missing, uploads go to the pod's local disk and vanish on restart, with no error surfaced to the user. There is no startup guard for this one.
3. **The Supabase service-role key must be the legacy JWT.** The newer `sb_secret_...` format fails against the Storage REST API with "Invalid Compact JWS".
4. **Use the transaction pooler, not the direct connection.** Supabase's direct host (`db.<ref>.supabase.co:5432`) is often IPv6-only on free projects.

5. **Free Supabase projects pause after about 7 days idle**, and deleted projects stop resolving in DNS entirely — which is exactly the state the configured project is in today.
6. **VITE_API_URL is baked in at build time**. It cannot be changed with a ConfigMap; changing the backend URL means rebuilding the frontend image.
7. **Do not set VITE_API_URL in a local frontend/.env**. It makes the local UI open its chat WebSocket against the deployed backend instead of the local one.
8. **The service name in the live URLs is spelled "contruction"** (missing the "s"). That is the real, working spelling — not a typo in this document.
9. **/docs is public** by default. Block it at the ingress if this becomes internet-facing.
10. **Do not add --workers N** as a quick scaling win. Each worker duplicates the in-memory WebSocket registry and the search index, giving the same split-brain behaviour as extra replicas plus $N \times$ the memory.
11. **The bundled dataset is fictional**. Everything under `backend/Database/`, `backend/company_data/` and `frontend/src/data/` is seeded demo content. No real personal data and no credential is in the repository.
12. **The frontend ships a 1.44 MB company-data chunk** (130 KB gzipped) bundled into the client. Not a bug, but it dominates first load on mobile if you ever profile it.

Quick reference

Question	Answer
Are there Dockerfiles?	Yes — <code>backend/Dockerfile</code> and <code>frontend/Dockerfile</code> . Neither has been built yet.
Is <code>Database/</code> the production DB?	No. Supabase Postgres is, via a single <code>documents</code> table.
Does <code>company_data/</code> serve from the container?	Committed demo assets yes (read-only, from the image). New uploads no — Supabase Storage.
Does <code>uploads/</code> take runtime writes?	Not in production. Supabase Storage does.
Is <code>index_data/</code> a rebuildable cache?	Yes, entirely. Per-pod, ephemeral, never shared.
EBS, S3, Supabase or ephemeral?	Ephemeral filesystem, with Supabase as the durable tier. No PVC required.
Deployment or StatefulSet?	Deployment . Start with one backend replica.
Container ports	Backend 8000 (reads <code>\$PORT</code>); frontend nginx 8080 as non-root.
Health endpoint	<code>GET /api/health</code>
Language versions	Python 3.11.9 (<code>runtime.txt</code>); Node 22 (<code>frontend/.nvmrc</code>).
Blocking issue before deploy	Create a new Supabase project — the configured one no longer exists.

All statements verified against the source on branch `main`, and by running both services locally against the real Supabase, Cohere and OpenRouter endpoints. Primary references: `backend/app.py`, `backend/backend/utils/{data_handler,db,storage,index_sync,events,semantic_embeddings}.py`, `backend/backend/routers/messages.py`, `backend/Dockerfile`, `backend/render.yaml`, `frontend/Dockerfile`, `frontend/nginx.conf.template`, `frontend/vercel.json`, `frontend/vite.config.ts`. The container images had not been built at the time of writing — no Docker daemon was available in the environment.